Step 3 — Containerized and Rate-Limited: Runs the Same Everywhere, Survives Abuse — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 3: the API works on your laptop, but it must run the same way on any machine, and survive a caller who sends 1,000 requests back to back — each one triggering five paid agent calls. A `Dockerfile` packages the app so it runs identically anywhere, with the API key passed in only at `docker run` time, never baked into the image. A rolling-window rate limiter checks the request count *before* any model is called and answers `429 Too Many Requests` once the window fills (Doc12's pattern). **If not:** "works on my machine" is all you could promise, and one script in a loop could empty your OpenAI budget in minutes.

After this step the project runs in a container: `docker build`, then `docker run` with the key passed in. Files not shown here don't change.

Solution — a limit per caller, a checked image, and a test for the 429

Changed from Step 2: `routes.py` gets Doc12's Build Task rate limiter, per caller (`http_request.client.host`), with its honest limitation written next to it, and checks it first in `POST /run-task`.

```
# src/api/routes.py
# The API routes. POST /run-task now checks a per-caller rate limit first
# (429 before any model call); the limiter is in-memory, so its limitation
# is written next to it.
import time                                                    # new

from fastapi import APIRouter, HTTPException

from src.api.schemas import TaskResponse
from src.db.database import get_connection, insert_started_run
from src.db.database import finish_run, fail_run, fetch_run
from src.db.models import Run
from src.pipeline import run_pipeline
from src.utils.logger import get_logger

router = APIRouter()
conn = get_connection()
```

```
# new in Step 3: the rate limiter
RATE_LIMIT = 5             # max requests per caller...
# LIMITATION: in-memory only. It resets on every restart, and each
# worker/container keeps its own separate counts — so 3 containers
# really allow 3 x RATE_LIMIT. A shared store (like Redis) fixes that.
request_times = {}         # caller -> list of request timestamps

INTERNAL_ERROR = "Internal error. Please try again later."

# new in Step 3
def check_rate_limit(caller_key):
    now = time.time()
    old_times = request_times.get(caller_key, [])
    recent = []
    for t in old_times:
        if now - t < 60:
            recent.append(t)
    if len(recent) >= RATE_LIMIT:
        raise HTTPException(status_code=429, detail="Rate limit exceeded")
    recent.append(now)
    request_times[caller_key] = recent

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/run-task", response_model=TaskResponse)
def run_task(request, http_request):
    logger = get_logger(__name__)
    request_id = http_request.state.request_id
    check_rate_limit(http_request.client.host)   # new in Step 3: checked first
    if not request.topic.strip():
        raise HTTPException(status_code=400, detail="topic cannot be empty")

    run_id = insert_started_run(conn, request.topic)
    logger.info(f"[{request_id}] run {run_id} started")
    try:
        outcome = run_pipeline(request.topic)
    except Exception:
        fail_run(conn, run_id)
        logger.exception(f"[{request_id}] run {run_id} failed")
        raise HTTPException(status_code=500, detail=INTERNAL_ERROR)

    log_text = "\n".join(outcome["steps"])
    finish_run(conn, run_id, outcome["article"], log_text)
    logger.info(f"[{request_id}] run {run_id} completed: {outcome['reason']}")
```

```
    return TaskResponse(
        run_id=run_id,
        status="completed",
        accepted=outcome["accepted"],
        reason=outcome["reason"],
        article=outcome["article"],
    )

@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id):
    run = fetch_run(conn, run_id)
    if run is None:
        raise HTTPException(status_code=404, detail="run not found")
    return run
```

Story — `Dockerfile`: this follows Doc12's shape, pointed at `src.api.main:app`. `requirements.txt` is copied and installed **before** `COPY . .`, so Docker's layer cache means a plain code change doesn't force the slow dependency install to rerun; `--host 0.0.0.0` (not the default `127.0.0.1`) is what makes the server reachable from outside the container at all. **If not:** every build would reinstall everything from scratch, and `curl` from the host would get "connection refused".

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
# requirements first — Docker reuses this slow layer when only code changed
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# brings main.py, src/ and data/ into the image
COPY . .
EXPOSE 8000
# --host 0.0.0.0, or nothing outside the container can reach it
CMD ["uvicorn", "src.api.main:app", \
     "--host", "0.0.0.0", "--port", "8000"]
```

Story — `.dockerignore`: this keeps `.env`, `.git`, the venv, and your local `runs.db` out of the built image entirely — a second, independent safety net under "secrets only at run time," in case someone forgets and the file is sitting in the build context anyway. **If not:** your key would sit baked into an image layer forever, visible to anyone with `docker history`.

`.dockerignore`

```
# a second safety net — even "COPY . ." can't copy your key
.env
```

```
.git
.venv/
__pycache__/
*.pyc
.pytest_cache/
# the local run history and eval report stay on your computer
runs.db
report.json
```

Changed from Step 2: the tests reset the limiter's memory in every test, and prove the 429.

```
# tests/api/test_routes.py
# The API's own behavior — status codes and replies — with Project 4's
# team FAKED: no model calls, no cost, same every time.
# Run from the repo root: pytest tests/api -v
# The app still refuses to start without OPENAI_API_KEY — any value
# works here, because no real model is ever called.
from fastapi.testclient import TestClient

from src.api import routes
from src.api.main import app

client = TestClient(app)

def fake_run_pipeline(topic):
    return {
        "article": "E-Bikes Explained\n\nUp to 100 km. [battery-review]",
        "accepted": True,
        "reason": "reviewer accepted the draft",
        "steps": [
            "research_agent: no research yet",
            "END: reviewer accepted the draft",
        ],
    }

def broken_run_pipeline(topic):
    raise RuntimeError("simulated internal failure")

def use_fake_team(monkeypatch, fake):
    monkeypatch.setattr(routes, "run_pipeline", fake)
    # new in Step 3: a fresh rate-limit memory per test
    monkeypatch.setattr(routes, "request_times", {})

def row_count():
    sql = "SELECT COUNT(*) FROM runs"
    return routes.conn.execute(sql).fetchone()[0]
```

```
def failed_count():
    sql = "SELECT COUNT(*) FROM runs WHERE status = ?"
    return routes.conn.execute(sql, ("failed",)).fetchone()[0]

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_valid_request_is_answered(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "completed"
    assert body["accepted"] is True
    assert "[battery-review]" in body["article"]

    saved = client.get("/runs/" + str(body["run_id"])).json()
    assert saved["input"] == "electric bikes"
    assert saved["status"] == "completed"
    assert "END: reviewer accepted the draft" in saved["log"]

def test_bad_body_is_422(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    before = row_count()
    response = client.post("/run-task", json={})
    assert response.status_code == 422
    assert row_count() == before

def test_blank_topic_is_400(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "   "})
    assert response.status_code == 400
    assert response.json() == {"detail": "topic cannot be empty"}

def test_internal_error_is_a_plain_500(monkeypatch):
    use_fake_team(monkeypatch, broken_run_pipeline)
    before = failed_count()
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 500
    expected = {"detail": "Internal error. Please try again later."}
    assert response.json() == expected
    assert failed_count() == before + 1

def test_unknown_run_is_404():
```

```
    response = client.get("/runs/999999")
    assert response.status_code == 404

def test_every_reply_has_a_request_id():
    response = client.get("/health")
    assert len(response.headers["X-Request-ID"]) == 8

# new in Step 3
def test_over_the_rate_limit_is_429(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    # a limit of 2 for this test only — quicker than sending 6
    monkeypatch.setattr(routes, "RATE_LIMIT", 2)
    statuses = []
    for i in range(3):
        response = client.post("/run-task", json={"topic": "solar panels"})
        statuses.append(response.status_code)
    assert statuses == [200, 200, 429]
```

Run it (from the repo root):

```
pytest tests/api -v
docker build -t contentforge-pro .
docker run -p 8000:8000 --env-file .env contentforge-pro
```

```
tests/api/test_routes.py::test_over_the_rate_limit_is_429 PASSED
8 passed
```

Then, from a second terminal:

```
$ curl localhost:8000/health
{"status":"ok"}
$ docker history --no-trunc contentforge-pro | grep "sk-"
(no output — the key is in no layer of the image)
$ docker run contentforge-pro
...
src.exceptions.MissingConfigError: Required environment
variable is missing: OPENAI_API_KEY
```

(That last error is really one line — shown wrapped.) Six quick `POST /run-task` calls from one machine: five are answered, the sixth gets `429 {"detail":"Rate limit exceeded"}`.

`--env-file .env` passes the key in when the container **starts**; the `.env` file itself never enters the image. The run history lives inside the container and is lost when the container is removed — fine for this project (see the README's Limitations).